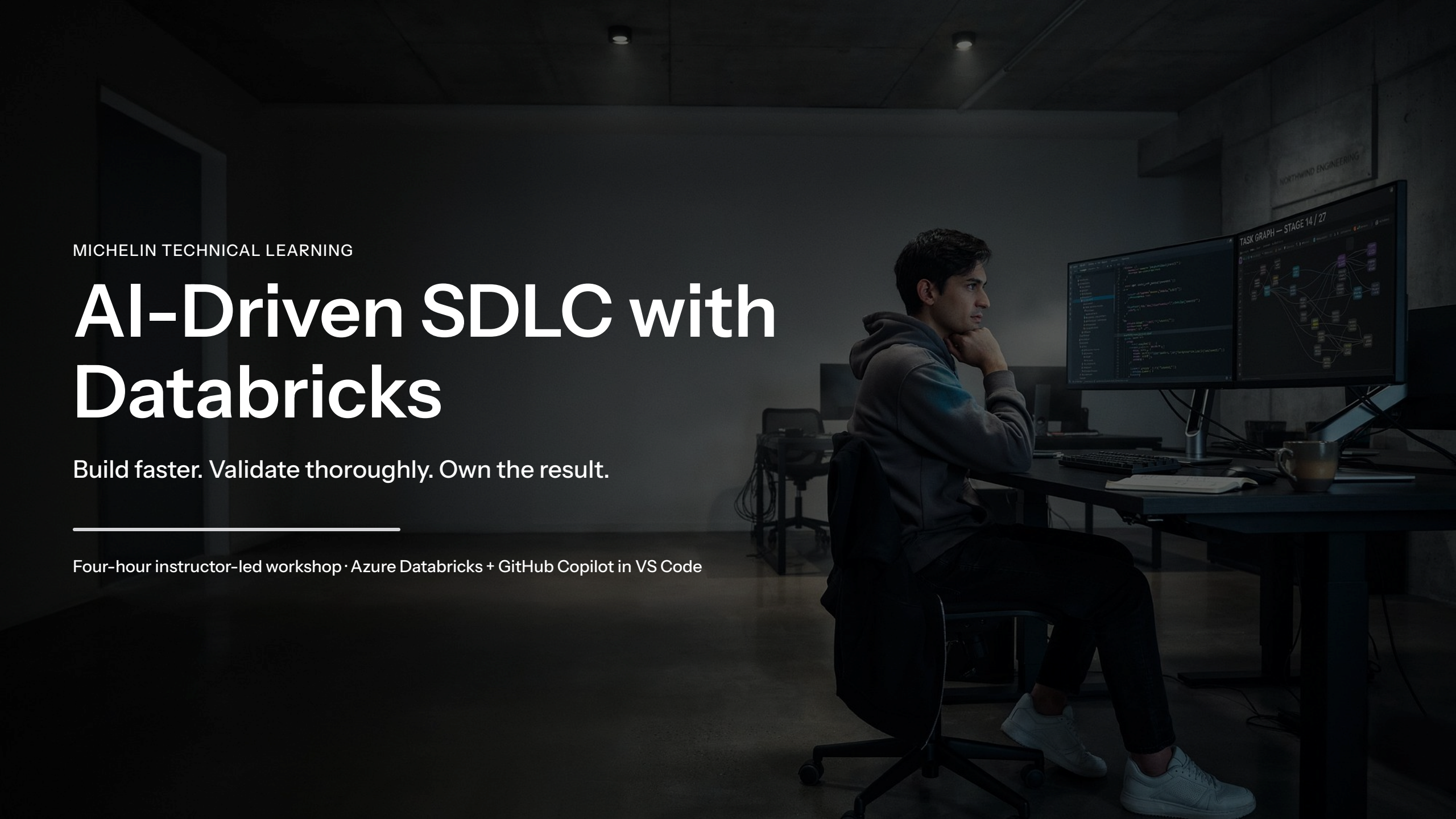
MICHELIN TECHNICAL LEARNING

# AI-Driven SDLC with Databricks

Build faster. Validate thoroughly. Own the result.

Four-hour instructor-led workshop · Azure Databricks + GitHub Copilot in VS Code



# Why we are here: productive, understandable, maintainable work

### Requirements to code

Turn clear requirements into code.

### Cross tools

Move confidently between Databricks and VS Code.

### Verify both

Check the implementation and the resulting data.

**Success is measured by verified results, not the volume of AI-generated code.**

# Who this is for and what you need

PRIMARY AUDIENCE

Product-squad software engineers already using Databricks for ETL

ALSO APPLIES TO

Platform and coaching engineers, data scientists, insight-focused developers

STARTING POINT

Read basic Python and SQL, recognize joins and data types, run an existing notebook

No prior VS Code or AI prompting expertise required. Orientation is built into the exercises.

# Six learning outcomes

1. Write a bounded AI request: business rule, schema, constraints, expected output, acceptance tests
2. Use Genie Code to generate, explain, and refine SQL/PySpark on approved training data
3. Use GitHub Copilot in VS Code to refactor reusable code, test locally where appropriate, and validate the same revision on Databricks
4. Create and run code-behavior and data-quality checks, including a failing test that exposes a seeded defect before repair
5. Explain and review AI-generated changes, flag unsafe or unsupported suggestions, keep human accountability
6. Choose the right tool for a daily task and prepare an evidence-based handoff to technical review

# Scope: what is in and what is out

CORE

Prompting, Python/SQL development, cross-tool working, debugging, tests, data-quality controls, human review

CONTEXT ONLY

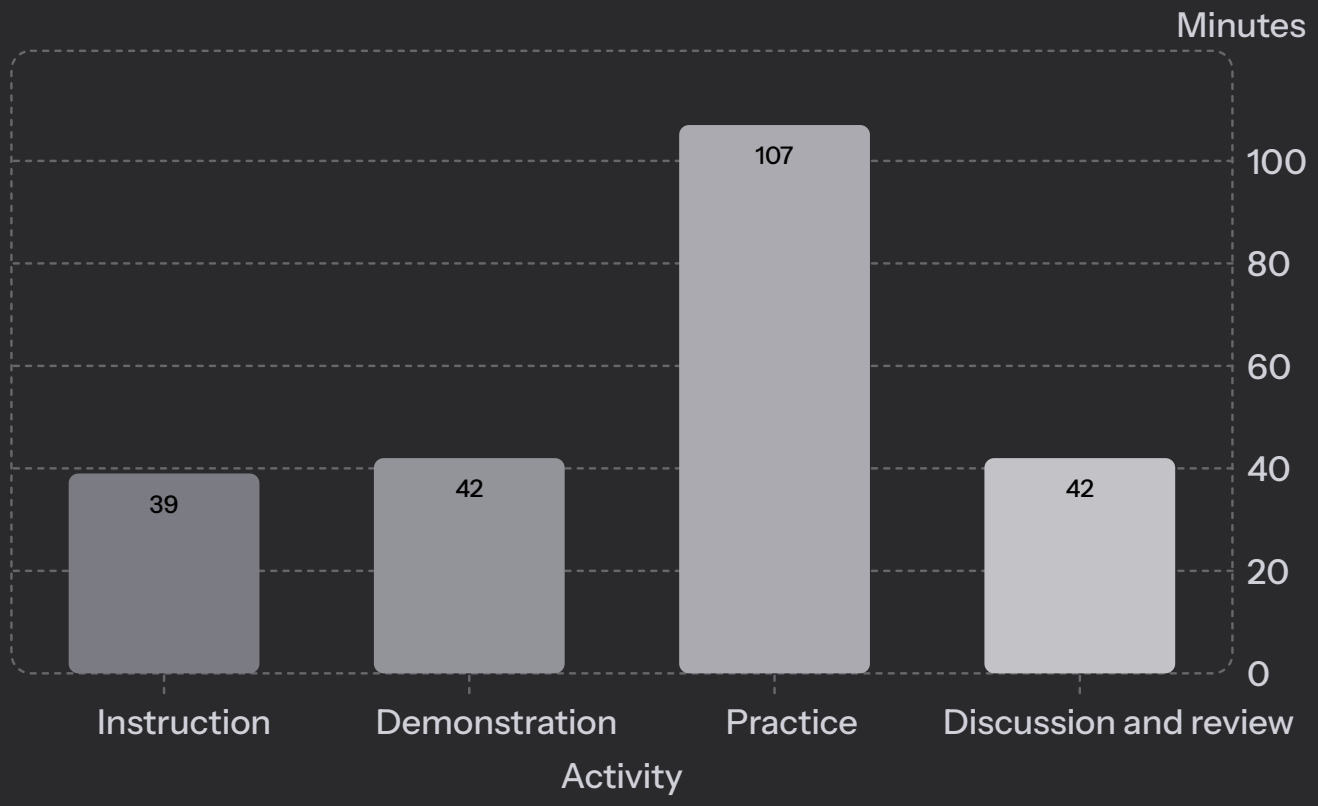
Azure Data Factory as orchestration; Azure DevOps remains the review workflow; production handoff is brief

OUT OF SCOPE

New CI/CD pipelines, platform deployment, advanced model training, agent construction

# Agenda: a four-hour learning journey

| Time | Module | Output |
| --- | --- | --- |
| 00:00-00:20 | 1 \| AI-assisted development with ownership | Improved prompt and review criteria |
| 00:20-01:05 | 2 \| Build and explain in Databricks | Reviewed transformation notebook |
| 01:05-01:50 | 3 \| Cross over to VS Code + Copilot | Reusable module and parity check |
| 01:50-02:00 | Break | |
| 02:00-03:00 | 4 \| Test the code and the data | Failing-to-passing test evidence |
| 03:00-03:45 | 5 \| Integrated change challenge | Validated change and peer explanation |
| 03:45-04:00 | 6 \| Apply the workflow every day | Tool choice and next-task commitment |




Learning mix in minutes

# 65%

## About 65% of learning time is hands-on

Practice (107 min) plus discussion and review (42 min) = 149 of 230 learning minutes. Instruction and demonstration make up the remaining 81.

230 min learning + 10 min break = 240 min

SHARED CASE

# Order-to-invoice quality review

- A sales analytics squad needs a trustworthy customer/product invoice dataset and an exception report
- You inherit a transformation that runs, yet can produce misleading totals
- Fictional, non-sensitive invoices, orders, customer and product master data: corporate data, not manufacturing telemetry

# Learning architecture

1

## Extracted input area

Invoices, orders, master data

2

## Azure Databricks transformation

Reviewed, tested code

3

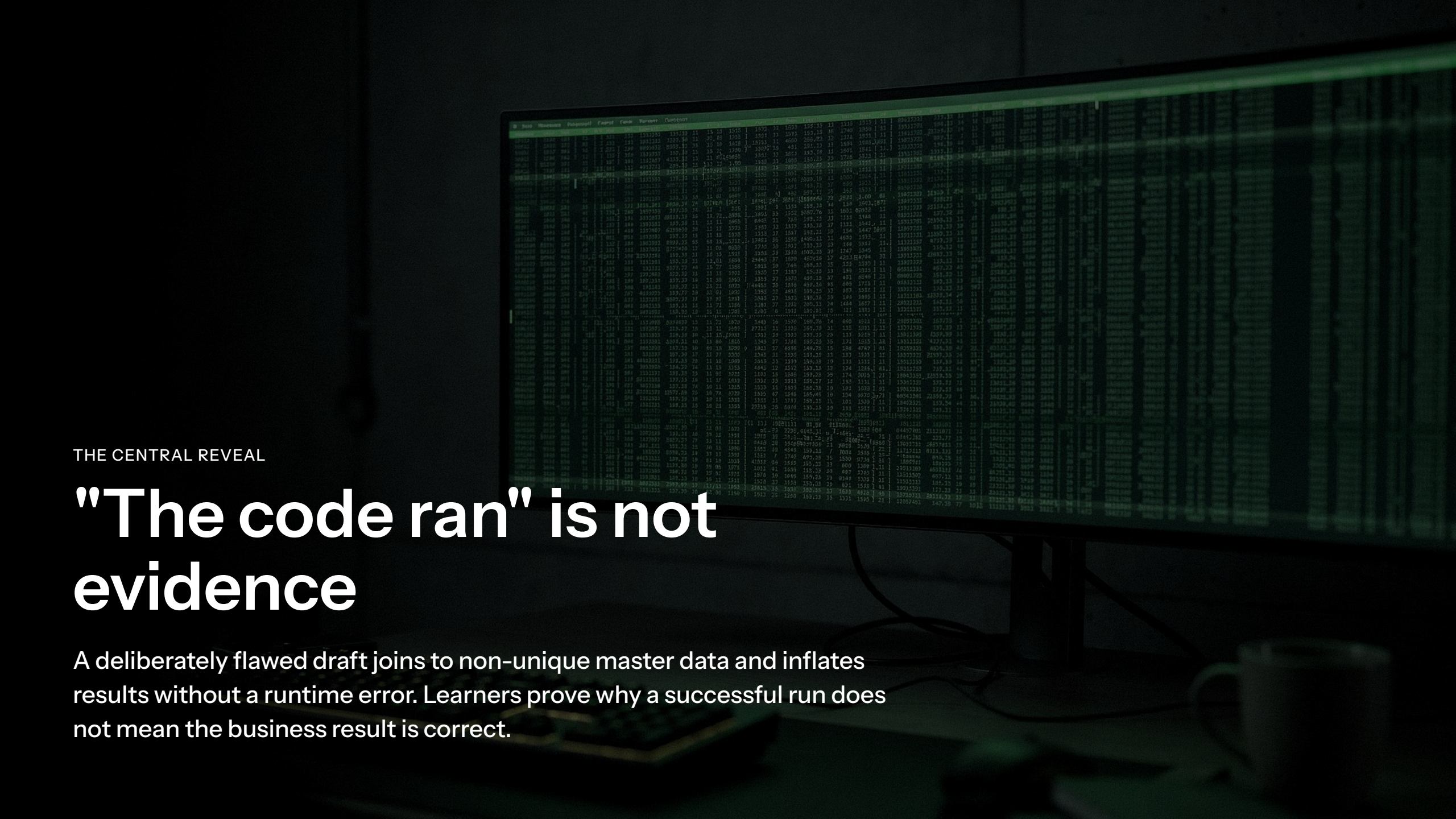
## Validated output

Plus exception evidence

Azure Data Factory shown as surrounding orchestration context, not rebuilt.

PROGRESSION

REVIEWED NOTEBOOK → REUSABLE PYTHON/PYSPARK MODULE → TESTED CHANGE PACKAGE



THE CENTRAL REVEAL

# "The code ran" is not evidence

A deliberately flawed draft joins to non-unique master data and inflates results without a runtime error. Learners prove why a successful run does not mean the business result is correct.

MODULE 1

# AI-assisted development with ownership

00:00–00:20

# A repeatable loop

AI is a development partner, not the owner of correctness or the final reviewer.

Tie every step to a real coding task.

00:00-00:20 · 8 min instruction, 5 min demo, 7 min paired prompt repair · Outcomes 1 and 5

# Prompt ingredients for a bounded request

Business intent

Input schema and grain

Expected result

Permitted libraries

Error behavior

Explicit acceptance criteria

Ask for assumptions before implementation; request small, reviewable changes.

# Weak prompt vs strong prompt

WEAK

"Clean this invoice data"

STRONG

- Names the table grain (InvoiceID + LineNumber)
- Keeps only posted lines
- Preserves signed amounts and decimal precision
- Routes unmatched customers/products to exceptions
- Uses no new libraries; lists assumptions first
- States the acceptance tests

# Responsible use is part of the workflow

### Approved only

Work only with approved code and data

### Clean prompts

Keep secrets and unapproved information out of prompts

### Inspect first

Inspect proposed commands, dependencies, and edits before execution

### Human review

Required throughout

EXERCISE · IN PAIRS

# Repair the prompt

01

## Rewrite

Turn "clean this invoice data" into a bounded development request.

02

## Challenge

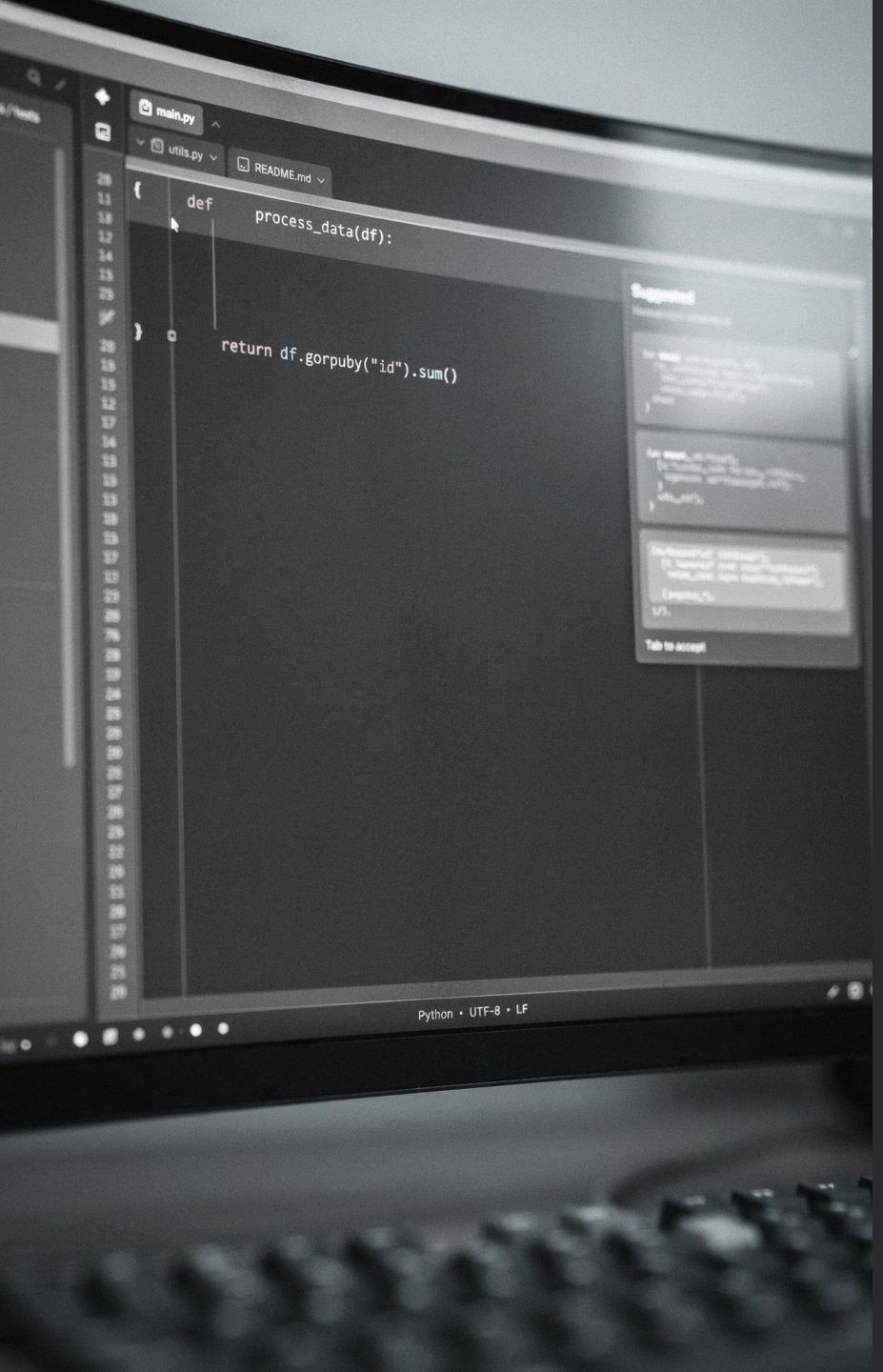
Partner identifies one missing business rule and one reason not to accept an AI answer immediately.

**Checkpoint:** each pair states what a correct result means and what evidence a reviewer will require. Save the prompt for Lab 1.

MODULE 2

# Generate, explain, and refine in Databricks

00:20-01:05




# Genie Code: the coding assistant inside Databricks

- Generates Python and SQL, explains code, diagnoses and fixes errors, refactors and optimizes
- Uses Unity Catalog metadata (tables, columns, descriptions, popular assets) to personalize responses; access is governed by your own permissions
- Available in notebooks, the SQL editor, the Lakeflow Pipelines Editor, and the file editor; use the side pane or inline in a cell (Ctrl+I on Windows, Cmd+I on macOS); reference tables and files with @
- Runs in Agent mode by default: it plans, runs code, reads cell outputs, and retries after errors
- Asks for approval before each tool use (run code, edit a notebook, query a table); in this course keep the setting at "Ask every time"
- Review every proposed edit before accepting; "Accept" does not run the code, "Accept and run" does

# Three products share the Genie name

## Genie Code

The AI coding and data assistant for developers inside the workspace. This is the tool used in the lab.

## Genie Agents

Formerly Genie Spaces (renamed July 2026). Curated datasets, metrics, and business rules that data teams configure. Not a notebook editor.

## Genie One

Formerly Genie (renamed June 2026). The chat experience for business users, answered by Genie Agents.

Genie Code is on by default. The partner-powered AI features setting no longer gates it and is removed from all workspaces on November 1, 2026.

⚠ Confirm in Michelin's sandbox before class: Genie Code is visible, Agent mode prompts for approval, and no Genie budget block applies to training users.

# Separate schema facts from AI assumptions

Inspect the preloaded invoice and master tables first

Compare the meaning of SQL and PySpark rather than trusting a translation

Review joins, types, signed amounts, and output grain before execution

00:20–01:05 · 8 min schema walkthrough, 12 min demo, 20 min Lab 1, 5 min debrief · Outcomes 1, 2, 5

DEMO

# From schema to a scoped transformation

01

## Inspect

Preloaded invoice and master tables

02

## Generate

A scoped SQL transformation

03

## Translate

One bounded operation to PySpark

04

## Review

Joins, types, signed amounts, and output grain before running

LAB 1

# Build the first reviewed transformation

1. Select posted invoice lines and preserve signed amounts
2. Enrich with customer and product attributes
3. Identify records that cannot safely enter the accepted output
4. Use the prepared rule sheet and expected-results fixture

# Quality from the first line of code

DO

- Verify invoice-line keys and row counts
- Inspect unmatched keys
- Compare selected SQL and PySpark results

DO NOT

- Silently fill unknown customers
- Discard conflicting duplicates
- Rewrite raw inputs

# Lab 1 output and checkpoint

OUTPUT

A runnable notebook, recorded assumptions, and a short explanation of join and filtering behavior

CHECKPOINT

Explain one generated expression and show a result that agrees with the independent fixture

REMAINING DEFECTS

Become a documented test target, not an ignored warning

MODULE 3

# Develop across tools without losing control

01:05-01:50

# Databricks code does not have to be built only in the browser

## Databricks

Execution and governed data access

## VS Code

Modular development, focused edits, and appropriate local tests

01:05–01:50 · 8 min tool map, 10 min refactoring demo, 22 min Lab 2, 5 min debrief · Outcomes 3, 5, 6

# What the Databricks IDE extension for VS Code does

- Connects local VS Code to a remote Databricks workspace
- Runs local Python files on clusters or serverless compute, or as Lakeflow Jobs
- Synchronizes local code to a workspace folder
- Sets up a local Python environment matched to the target compute and installs a compatible Databricks Connect

Databricks Connect requires a Unity Catalog-enabled workspace and supported compute.

# Editing locally does not make Spark local

- With Databricks Connect, Spark processing remains on remote Databricks compute
- Editing in VS Code does not make that compute free or local
- Use Michelin-approved routes, entitlements, and metering

# Demo and Lab 2: refactor, test, run, compare

01

## Orient and refactor

Copilot extracts one helper and a transformation module with a thin entry point; review the diff, reject unnecessary dependencies

02

## Test locally

Small non-Spark Python tests on approved fixtures with the supplied scaffold

03

## Execute remotely

Same revision on the Michelin sandbox via the prevalidated IDE workflow

04

## Compare

Schemas, key sets, row counts, signed decimal amounts, selected aggregates

# Matching a grand total is not proof of equivalence

Two results can share a total while differing in rows, keys, or signs.

SCHEMAS | INVOICE-LINE KEY SETS | ROW COUNTS | SIGNED DECIMAL AMOUNTS | SELECTED AGGREGATES

Parity means all of these agree.

# Task-to-tool decision framework

| Task | Starting choice | Evidence / boundary |
| --- | --- | --- |
| Understand a table or debug a notebook result | Databricks + Genie Code | Review actual schema and execution output |
| Extract a helper or improve maintainability | VS Code + Copilot | Inspect a small diff and run local unit tests |
| Validate Spark joins and governed data access | Approved Databricks compute | Run integration and data-quality checks |
| Accept a change for technical review | Human developer / reviewer | Explain behavior, tests, risks, limitations |

# One source of truth for the code

1

### Scratch

Lab 1 prototype stays in a separate scratch location

2

### Local project

After extraction, the version-controlled project owns the code

3

### Databricks mirror

IDE folder sync is one-way

Never edit the remote mirror and expect changes to flow back.

# Lab 2 output, checkpoint, and cost discussion

### Output

Reusable module, thin execution wrapper, local test result, parity check tied to one revision

### Checkpoint

Name what ran locally and what ran on Databricks

### Cost-aware

Genie Code is metered per user in DBUs (150 free per month, pay-as-you-go above that). Copilot is a per-seat license. Compare context quality, execution needs, AI usage, and rework; neither tool is always cheaper.

# Break - 10 minutes

MODULE 4

# Test the implementation and its business result

02:00-03:00

# Code tests and data-quality checks answer different questions

CODE TESTS

**Does a function behave as specified?**

DATA-QUALITY CHECKS

**Do records, relationships, and outputs satisfy the business contract?**

AI proposes cases and fixes; you verify expected answers independently.

02:00-03:00 · largest module: 10 min model, 15 min demo, 30 min Lab 3, 5 min debrief

# Core test matrix

| Check | Seeded problem | Required evidence |
| --- | --- | --- |
| Behavior and precision | Helper mishandles a null or changes a signed credit | Unit test verifies behavior and decimal precision |
| Schema and required fields | Missing key, invalid date, unexpected type | Explicit field check and traceable exception |
| Uniqueness and replay | Exact replay vs conflicting rows with the same key | Documented replay handling; no arbitrary discards |
| Relationships and joins | Unknown customer/product or duplicate master key | Referential and join-cardinality checks |
| Business rules | Non-posted invoice accepted or valid credit rejected | Rule tests on status and sign |
| Reconciliation and regression | Repair loses records or drifts across tools | All rows accounted for; fixture, parity, regression pass |

# Databricks-native data-quality controls: pipeline expectations

In Lakeflow pipelines (formerly Delta Live Tables), an expectation is a named SQL Boolean constraint evaluated for every record written to a materialized view, streaming table, or view.

ON VIOLATION

### Warn (default)

```
CONSTRAINT valid_amount
EXPECT (amount IS NOT NULL)
```

Invalid rows are written to the target; violation metrics are logged.

### Drop

```
CONSTRAINT valid_customer
EXPECT (customer_id IS NOT
NULL) ON VIOLATION DROP
ROW
```

Invalid rows are dropped before the write.

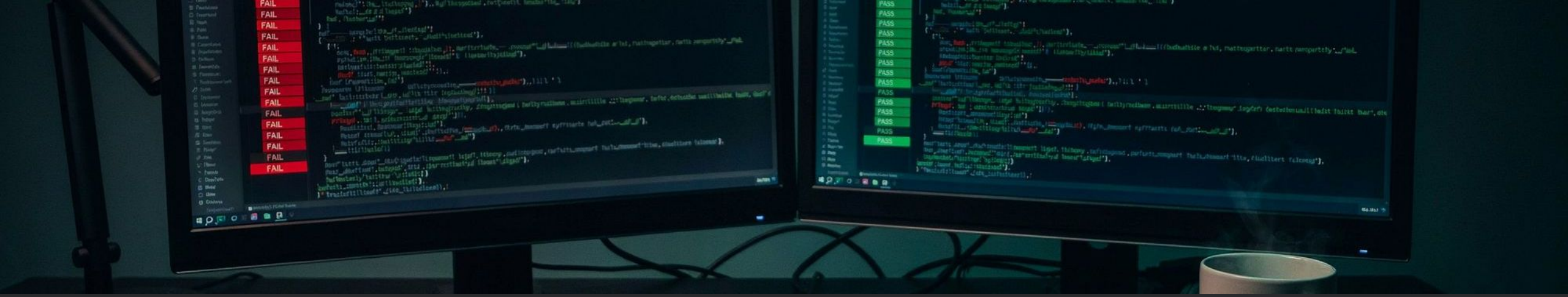
### Fail

```
CONSTRAINT posted_only
EXPECT (status = 'POSTED') ON
VIOLATION FAIL UPDATE
```

The update fails; manual intervention is required before reprocessing.

Python equivalents: @dp.expect, @dp.expect_or_drop, @dp.expect_or_fail

Reference for production patterns. The labs use supplied check scaffolds, not new pipelines.




# Lab 3: make the failure visible, then repair it

1. Inspect the rule sheet; ask AI for missing test cases
2. Complete at least two code tests and three data-quality checks from scaffolds
3. Run the failing checks before changing the implementation
4. Trace one failure to specific records and one to a code decision
5. Request a minimal fix, inspect it, rerun affected checks, then the full regression suite
6. Retain raw data; record every quarantine or exclusion reason

# Acceptance discipline

NEVER

Delete, skip, or weaken a failing assertion to get a passing run

AI MUST NOT

Invent expected totals or redefine the business rule

ALWAYS

Compare against a small hand-verifiable fixture and the instructor-validated answer set

# Lab 3 output and checkpoint

OUTPUT

Executable test set, before/after failure evidence, exception summary with record keys

CHECKPOINT

Each pair explains why its fix corrects the cause rather than hiding the symptom

REMEMBER

A passing unit test alone does not authorize acceptance of the dataset

MODULE 5

# Integrated change challenge

03:00-03:45

THE BUSINESS CHANGE

# Validate every invoice line against an order line

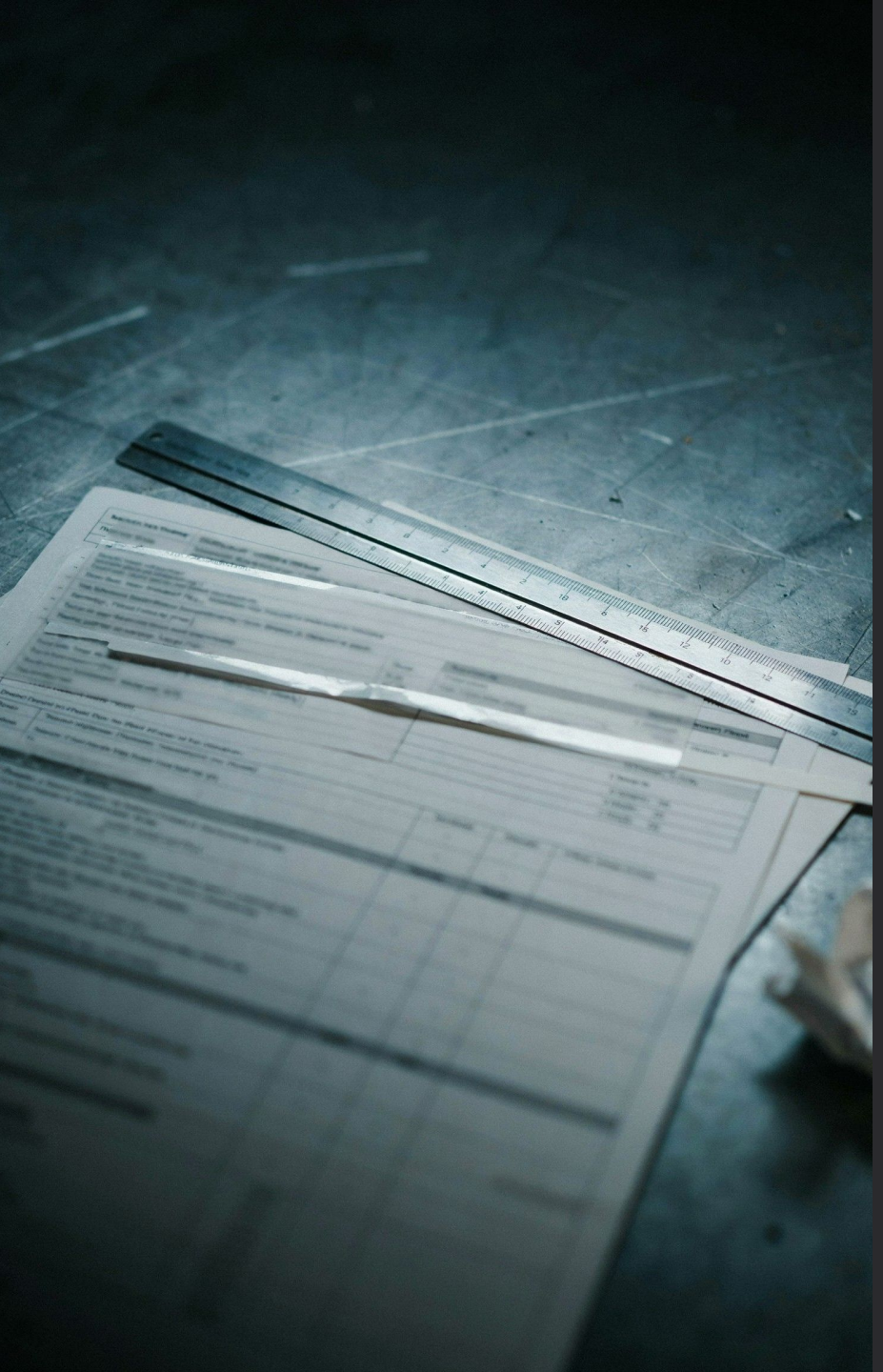
### Missing or ambiguous references

Reported, not guessed or silently dropped

### Valid posted credit lines

Keep their signed values and stay eligible

03:00–03:45 · 5 min briefing, 28 min Lab 4, 12 min paired review · Outcomes 1–6



LAB 4

# Extend without regression

- Start from the verified Lab 3 checkpoint
- Define new acceptance cases, choose where to work, make a bounded AI-assisted change
- Validate on the internal Databricks sandbox

**Challenge batch:** an unknown order reference, an exact replay, a valid signed credit. Prior controls must still work.

# Peer review: explain it without reading the AI answer

01

Developer gives a two-minute explanation of the change and its evidence

02

Reviewer asks one business-rule question and one test question

03

Close one gap, then switch roles

# Performance check: all five conditions must be met

### Correctness

The order-reference rule matches the expected cases

### Regression protection

A test shows the new failure; repaired and existing tests pass

### Data accountability

No silent loss or fan-out; accepted and exception results reconcile

### Cross-tool fidelity

The reviewed revision is the one validated on Databricks

### Human understanding

Behavior and limitations explained without reading an AI answer

Otherwise: "needs rework," not "review ready." A learning check, not release approval.

MODULE 6

# Apply the workflow every day

03:45-04:00

# Choose the tool, prepare the handoff

4 MIN

## Choose the tool

Place your next real task in Databricks, VS Code, or a deliberate combination; justify by context, execution needs, maintainability, cost.

4 MIN

## Prepare the handoff

Purpose, code revision, changed behavior, test/data evidence, remaining risks, questions for the tech lead.

Keep Michelin's Azure DevOps and human review process; no new deployment pipeline.

# Commit to your next task

One task to try

A reusable prompt

The required quality check

The intended human reviewer

Measure time-to-verified-result and rework, not generated line count.

# Reusable takeaways

Prompt pattern

Code and data review checklist

Tool-selection guide

Review-note template for the existing workflow

# AI may accelerate the work; the developer must understand and validate the result

Final questions and close.